

Hibernate & JPA

Q1. A developer maps an entity class but forgets to annotate any field with @Id. What happens when the application starts?

- A) Hibernate auto-generates a surrogate primary key
- B) The application fails to start with a mapping exception
- C) The entity is treated as an embeddable and saved without a key
- D) Hibernate silently uses the first field as the primary key

Q2. A REST controller calls repository.findById() inside a @Transactional service, returns the entity, and the view layer (outside the transaction) later accesses a lazy @OneToMany collection on it. What is the most likely outcome?

- A) The collection loads fine because Hibernate caches it globally and locally
- B) LazyInitializationException because the session is already closed
- C) The collection is automatically converted to eager on access
- D) A NullPointerException is thrown instead

Q3. Within a single transaction, a service calls session.get(Employee.class, 1) twice. How many SQL SELECT statements does Hibernate execute for the second call?

```
Employee e1 = session.get(Employee.class, 1);
Employee e2 = session.get(Employee.class, 1);
```

- A) Two — Hibernate always hits the database and cache memory.
- B) Zero — the second call is served from the first-level cache
- C) One, but only if second-level cache is enabled
- D) It depends on the isolation level of the transaction

Q4. A team wants: when a department is deleted, all its employee records should also be deleted, but persisting a new Employee should NOT automatically persist an unsaved Department. Which cascade type fits best?

- A) CascadeType.ALL or CascadeType.REFRESH
- B) CascadeType.REMOVE
- C) CascadeType.PERSIST
- D) CascadeType.REFRESH

Q5. Fetching 100 Orders and then accessing order.getItems() for each inside a loop produces 1 query for orders plus 100 queries for items. What is this problem called and what is a standard fix?

```
List<Order> orders = orderRepo.findAll();
for (Order o : orders) {
    o.getItems().forEach(System.out::println);
}
```

- A) Dirty checking problem — fix with both @DynamicUpdate and @EntityGraph
- B) N+1 select problem — fix with JOIN FETCH or @EntityGraph
- C) Cache eviction problem — fix with @Cacheable
- D) Deadlock problem — fix with pessimistic locking

Q6. In a bidirectional @OneToMany/@ManyToOne mapping, the Employee side is annotated @ManyToOne @JoinColumn(name="dept_id"), and the Department side has @OneToMany(mappedBy = "department"). What does mappedBy indicate?

- A) Department owns the one and two way relationship and controls the foreign key
- B) Employee is the owning side; Department is the inverse (non-owning) side
- C) Both sides equally own the foreign key column
- D) mappedBy creates a separate join table

Q7. A report needs a complex aggregation with database-specific functions not expressible cleanly in HQL/JPQL. Which approach is most appropriate?

- A) Force it into HQL using only entity names and) Loop over all entities in Java and aggregate manually
- B) Use a native SQL query via @Query(nativeQuery = true) or createNativeQuery
- C) Loop over all entities in Java and aggregate manually
- D) Disable the second-level cache instead

Spring Core & Boot

Q8. A Spring bean holds mutable per-request state and is injected into a singleton controller without changing its scope. What problem arises under concurrent requests?

- A) No problem — Spring isolates state automatically and share mutable state.
- B) Shared mutable state causes race conditions across requests
- C) The application fails to start
- D) Spring converts it to prototype scope automatically

Q9. Two beans of type PaymentGateway (StripeGateway and PaypalGateway) exist, and a class autowires PaymentGateway by type without qualification. What happens at startup?

```
@Autowired
private PaymentGateway gateway;
```

- A) Spring picks the first one alphabetically and second randomly.
- B) NoUniqueBeanDefinitionException is thrown
- C) Both beans are merged into one
- D) Spring picks the most recently defined bean

Q10. BeanA depends on BeanB via constructor injection, and BeanB depends on BeanA via constructor injection. What does Spring do?

- A) Resolves it lazily without issue
- B) Throws BeanCurrentlyInCreationException for a circular dependency
- C) Creates two separate instances of each bean
- D) Automatically switches to setter injection

Q11. A service method calls another @Transactional method on 'this' from within the same class. The transactional behavior on the inner call does not apply. Why?

```
@Service
```

```
public class OrderService {
    public void placeOrder() {
        this.saveOrder(); // self-invocation
    }
    @Transactional
    public void saveOrder() { ... }
}
```

- A) Transactional annotations only work on interfaces, never classes and abstract classes.
- B) Spring's proxy-based AOP is bypassed on self-invocation, so the advice never runs
- C) The method name must start with 'save' to be proxied
- D) @Transactional requires XML configuration to work at all

Q12. A Spring Boot app auto-configures a DataSource, but the team wants to supply their own custom DataSource bean and stop Boot's default from being created. What is the cleanest way?

- A) Delete the spring-boot-starter-data-jpa dependency entirely
- B) Define a custom @Bean of type DataSource; Boot's conditional auto-configuration backs off
- C) Set spring.datasource.enabled=delete in application.properties and Boot's conditional auto-configuration backs on
- D) Annotate the main class with @NoAutoConfig

Q13. A class contains business logic that talks only to other services (no direct DB or web layer access). Which stereotype annotation is most semantically correct?

- A) @Repository
- B) @Controller
- C) @Service
- D) @Configuration

Q14. The team needs different database URLs for 'dev' and 'prod' environments without changing code. Which Spring feature is designed for this?

- A) @ConditionalOnClass
- B) Spring Profiles with profile-specific application-{profile}.properties files
- C) Multiple main() methods
- D) @Scope("environment")

REST APIs

Q15. A client calls the same PUT /accounts/5 request three times with identical payload. The resulting account state is the same after each call. This property is called:

- A) Statelessness
- B) Idempotency
- C) Cacheability
- D) Content negotiation

Q16. A client sends a GET request for /orders/999 where order 999 does not exist. Which HTTP status code is most appropriate?

- A) 400 Bad Request
- B) 204 No Content
- C) 404 Not Found
- D) 500 Internal Server Error

Q17. In a Spring REST controller, JSON data sent in the HTTP body needs to be mapped to a Java object, while a query string filter parameter needs to be read separately. Which pair of annotations is correct?

```
@PostMapping("/orders")
public Order create(@_____ Order order, @_____ String status) { ... }
```

- A) @RequestParam, @RequestBody
- B) @RequestBody, @RequestParam
- C) @PathVariable, @RequestBody
- D) @ModelAttribute, @PathVariable

Q18. An API response for a single order includes links to related resources like 'cancel' and 'payment' so clients can discover available actions dynamically. This is an example of:

- A) gRPC reflection
- B) HATEOAS
- C) GraphQL introspection
- D) OAuth2 discovery

Q19. A public API has been live for two years and needs a breaking change to its response shape without disrupting existing clients. What is the most standard REST practice?

- A) Silently change the response and notify clients by email
- B) Introduce a new versioned endpoint, e.g. /v2/orders, alongside /v1/orders
- C) Change only the status codes and keep the body identical
- D) Force all clients to upgrade within 24 hours

Q20. Multiple controllers in a Spring Boot app throw various exceptions, and the team wants one centralized place to convert exceptions into consistent JSON error responses. Which annotation enables this?

- A) @ExceptionHandler on every controller individually combine with @ControllerAdvice
- B) @ControllerAdvice combined with @ExceptionHandler methods
- C) @ResponseStatus on the exception class only
- D) @RestController alone handles this automatically

Q21. In a Eureka-based architecture, the Eureka server goes down temporarily. What happens to services that already fetched their registry cache?

- A) All services immediately crash
- B) Services can still communicate using their locally cached registry
- C) All inter-service calls are permanently blocked and client-side caching
- D) Services automatically switch to a hardcoded IP list

Q22. The Loan Service occasionally calls a slow, failing downstream Account Service, and repeated calls risk cascading failure across the system. Which pattern directly addresses this?

- A) Circuit Breaker
- B) Service Discovery
- C) API Gateway aggregation
- D) Database sharding

Q23. A team wants a single entry point that routes /accounts/ to the Account Service and /loans/** to the Loan Service, while also handling cross-cutting concerns like auth and rate limiting. What component provides this?**

- A) Config Server
- B) API Gateway
- C) Message Broker
- D) Service Registry

Q24. An order placement spans the Order Service and Inventory Service, each with its own database, and there is no shared distributed transaction. Which pattern manages consistency across these steps using compensating actions on failure?

- A) Two-Phase Commit (2PC)
- B) Saga pattern
- C) CQRS only
- D) Optimistic locking

Q25. Ten microservices each need environment-specific properties, and the team wants to manage all of them centrally with the ability to refresh config without redeploying. Which Spring Cloud component fits?

- A) Spring Cloud Config Server
- B) Spring Cloud Gateway
- C) Eureka Server
- D) Spring Cloud Sleuth

Q26. Service A needs an immediate response from Service B to continue processing the current request. Which communication style is appropriate?

- A) Asynchronous messaging via a queue
- B) Synchronous REST/Feign call
- C) Fire-and-forget event publishing
- D) Batch nightly ETL job

Q27. SonarQube flags a piece of code as having high cyclomatic complexity and duplicated logic, but it does not cause incorrect behavior. How is this classified?

- A) Bug
- B) Vulnerability
- C) Code Smell
- D) Security Hotspot

Q28. A pull request fails the CI pipeline because new code coverage dropped below the configured threshold on 'New Code'. What just failed?

- A) The Quality Gate
- B) The unit test runner itself
- C) The Docker build
- D) The Git merge

Q29. SonarQube reports several blocks of near-identical code copy-pasted across three service classes. Which metric captures this directly?

- A) Cognitive Complexity
- B) Duplicated Lines (%)
- C) Coverage
- D) Reliability Rating

Q30. A code snippet concatenates raw user input directly into a SQL query string. How would SonarQube most likely classify this finding?

```
String query = "SELECT * FROM users WHERE name = '" + userInput + "'";
```

- A) Code Smell only
- B) Vulnerability / Security Hotspot (SQL Injection risk)
- C) Duplication
- D) Cognitive Complexity issue

Q31. Technical Debt Ratio in SonarQube is best described as:

- A) The percentage of code lines written by junior developers
- B) The estimated cost to fix all issues, relative to the cost of developing the code from scratch
- C) The number of unit tests missing
- D) The number of open GitHub issues

Q32. A team wants generated code (e.g. DTOs from a code generator) to be excluded from SonarQube analysis so it doesn't skew metrics. Which mechanism is correct?

- A) Delete the generated files before every commit and make the project as maven update
- B) Configure sonar.exclusions / sonar.coverage.exclusions in the project settings
- C) Mark every generated class @Deprecated
- D) There is no way to exclude files in SonarQube

Mockito

Q33. In a unit test, OrderService depends on OrderRepository. The test wants a fake repository injected automatically into a real OrderService instance. Which annotation pair is correct?

```
@Mock
private OrderRepository orderRepository;
```

```
@_____
private OrderService orderService;
```

- A) @Spy
- B) @InjectMocks
- C) @Captor
- D) @Autowired

Q34. A test needs to stub a void method, deleteById(), to throw an exception when called. Which Mockito syntax is correct?

- A) when(repo.deleteById(1)).thenThrow(new RuntimeException())
- B) doThrow(new RuntimeException()).when(repo).deleteById(1)
- C) verify(repo).deleteById(1).thenThrow(new RuntimeException())
- D) mock(repo).deleteById(1).throwException()

Q35. A test needs to assert that repository.save() was called exactly twice during a method execution. Which line achieves this?

- A) assertEquals(2, repo.save());
- B) verify(repo, times(2)).save(any());
- C) verify(repo).save(any(), 2);
- D) assertCalls(repo, "save", 2);

Q36. A test wants to inspect the exact Order object passed into repository.save(order) to verify its fields, without knowing it in advance. Which Mockito feature fits?

- A) ArgumentMatchers.eq()
- B) ArgumentCaptor
- C) @Spy
- D) RETURNS_DEEP_STUBS

Q37. A developer wants a mock that calls the real method implementation by default, but selectively overrides only one method's behavior. Which Mockito construct is designed for this?

- A) @Mock
- B) @Spy
- C) @InjectMocks
- D) @Captor

Q38. In a stub, when(repo.findById(1)).thenReturn(Optional.of(employee)) is written, but the test calls repo.findById(2). What does Mockito return?

- A) The same employee object regardless of ID
- B) Optional.empty() by default, since no stub matches that argument
- C) A NullPointerException and throws UnnecessaryStubbingException immediately
- D) It throws UnnecessaryStubbingException immediately

JUnit

Q39. A test class needs an expensive resource (e.g. a database connection) created once for all tests, not before every single test method. Which annotation is correct in JUnit 5?

- A) @BeforeEach (on a Nonstatic method)
- B) @BeforeAll (on a static method)
- C) @Test
- D) @AfterEach

Q40. A team wants to run the same test logic against five different input/output pairs without duplicating the method. Which JUnit 5 feature is designed for this?

- A) @RepeatedTest with @ValueSource/@CsvSource
- B) @ParameterizedTest with @ValueSource/@CsvSource
- C) @Nested with @ValueSource/@CsvSource
- D) @Tag with @ValueSource/@CsvSource

Q41. What is wrong with this assertion given actual = 5 and expected = 4?

```
assertEquals(actual, expected);
```

- A) Nothing — assertEquals ignores argument order assertEquals cannot compare integers and String value
- B) The arguments are reversed; JUnit convention is assertEquals(expected, actual), which affects failure message clarity
- C) assertEquals cannot compare integers
- D) It should be assertTrue instead

Q42. A method is expected to throw IllegalArgumentException when given null input. Which JUnit 5 style correctly verifies this?

- A) @Test(expected = IllegalArgumentException.class)
- B) assertThrows(IllegalArgumentException.class, () -> service.process(null));
- C) assertEquals(IllegalArgumentException.class, service.process(null));
- D) try { service.process(null); } catch (Exception e) { assertTrue(true); }

Q43. Two tests in the same class unintentionally rely on execution order and both modify a shared static field, causing flaky results. What is the underlying best-practice violation?

- A) Tests should never use assertions and not shared mutable state across tests should be avoided or reset
- B) Tests should be independent and isolated; shared mutable state across tests should be avoided or reset
- C) JUnit does not support static fields at all
- D) Tests must always run in alphabetical order

Q44. A flaky integration test needs to be temporarily skipped without deleting it, with a documented reason. Which JUnit 5 annotation is correct?

- A) @Ignore("flaky")
- B) @Disabled("flaky - ticket JIRA-123")
- C) @Skip
- D) @Test(skip = true, ("flaky - ticket JIRA-123"))

React

Q45. A button's onClick handler reads a useState value but keeps logging an old value even after several clicks update the state. What is the most likely cause?

```
const [count, setCount] = useState(0);
function handleClick() {
  setTimeout(() => {
    console.log(count);
  }, 3000);
}
```

- A) useState is broken in this React version and the closure captured the count value from the render when handleClick was created (stale closure)
- B) The closure captured the count value from the render when handleClick was created (stale closure)
- C) setTimeout always returns 0
- D) console.log cannot read state variables

Q46. A useEffect that fetches data runs on every single render, causing excessive API calls. What is missing?

```
useEffect(() => {
  fetchData();
});
```

- A) Nothing is missing; this is correct usage
- B) A dependency array, e.g. useEffect(() => { fetchData(); }, []) to control when it re-runs
- C) useEffect should be replaced with useState
- D) fetchData must be called inside useState

Q47. A child component receives a 'username' value from its parent and must not modify it directly. This value is best described as:

- A) State, because it changes
- B) A prop, which is read-only from the child's perspective
- C) A ref, because it persists across renders which is read-only from the parent perspective
- D) Context, because it is global which is read-only from the child's perspective

Q48. A list is rendered using the array index as the React key, and reordering the list causes UI glitches (wrong items highlighted). What is the recommended fix?

```
{items.map((item, index) => (  
  <li key={index}>{item.name}</li>  
))}
```

- A) Remove the key prop entirely
- B) Use a stable, unique identifier from the data (e.g. item.id) as the key instead of the index
- C) Switch from map() to forEach() and Use a stable, unique identifier as the key instead of the index
- D) Wrap the list in a Fragment

Q49. An <input> element's value is driven entirely by React state and updated via onChange. This pattern is called:

- A) An uncontrolled component
- B) A controlled component
- C) A pure component
- D) A memoized component

Q50. A component should render a 'Loading...' message while data is null, and the actual content once data arrives. Which snippet correctly implements this?

```
return (  
  <div>  
    {data ? <Content data={data} /> : <p>Loading...</p>}  
  </div>  
);
```

- A) This is broken JSX syntax
- B) This correctly uses a ternary for conditional rendering
- C) This will render both Content and Loading simultaneously
- D) The ternary operator cannot be used inside JSX

Answer Key

Hibernate & JPA

Q1: B	Q2: B	Q3: B	Q4: B	Q5: B	Q6: B
Q7: B					

Spring Core & Boot

Q8: B	Q9: B	Q10: B	Q11: B	Q12: B	Q13: C
Q14: B					

REST APIs

Q15: B	Q16: C	Q17: B	Q18: B	Q19: B	Q20: B
--------	--------	--------	--------	--------	--------

Microservices

Q21: B	Q22: A	Q23: B	Q24: B	Q25: A	Q26: B
--------	--------	--------	--------	--------	--------

SonarQube

Q27: C	Q28: A	Q29: B	Q30: B	Q31: B	Q32: B
--------	--------	--------	--------	--------	--------

Mockito

Q33: B	Q34: B	Q35: B	Q36: B	Q37: B	Q38: B
--------	--------	--------	--------	--------	--------

JUnit

Q39: B	Q40: B	Q41: B	Q42: B	Q43: B	Q44: B
--------	--------	--------	--------	--------	--------

React

Q45: B	Q46: B	Q47: B	Q48: B	Q49: B	Q50: B
--------	--------	--------	--------	--------	--------